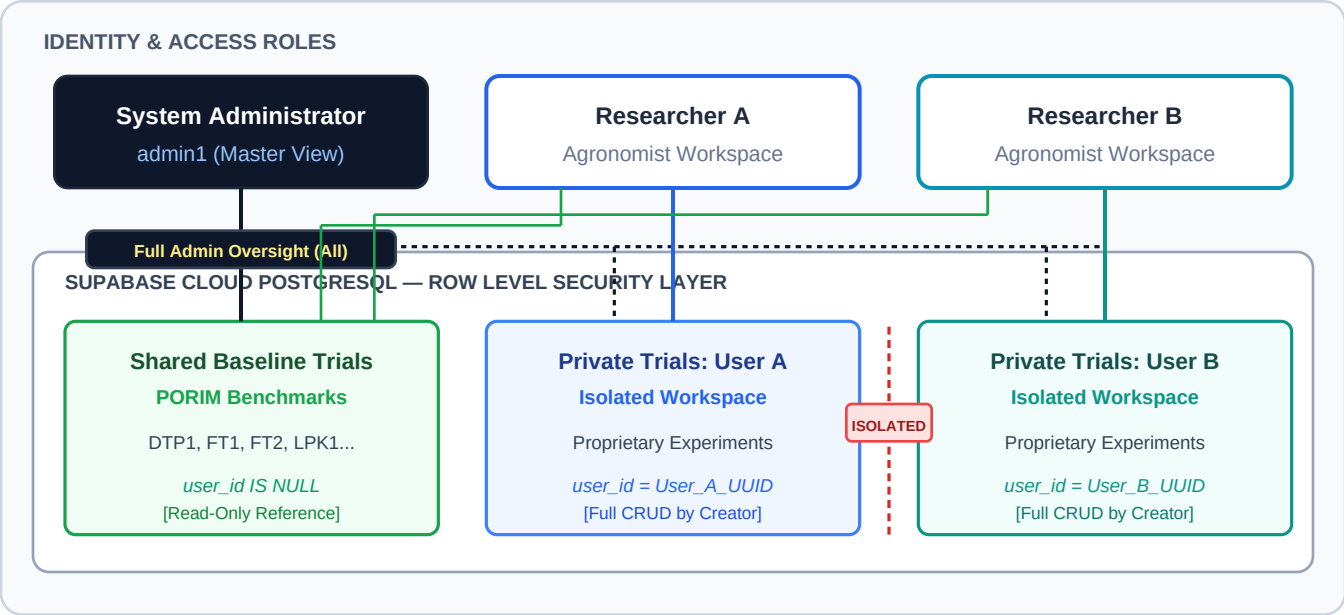
PALMNEX RESEARCH AND DATABASE SYSTEM (PALMNEXREADS)

Technical Architecture & Access Control Specification

Hybrid Multi-Tenant Data Isolation with Administrative Oversight (Option 3)

1. Executive Overview

This specification establishes the target database architecture and Row-Level Security (RLS) policies for PalmnexReaDS on Supabase PostgreSQL. Under the **Hybrid Multi-Tenant Model (Option 3)**, the platform provides two co-existing layers: a **Shared Baseline Benchmark** repository accessible station-wide, alongside **Isolated Personal Workspaces** for individual researchers, supervised by an **Administrative Role (`admin1`)** with master visibility.



2. Identity Roles & Access Hierarchy

Role Identifier	Access Scope	Shared Baseline Trials	Personal Trials	Peer Trials (Other Users)
admin1 System Administrator	Global / Station-Wide	**Full CRUD** (Read, Edit, Manage)	**Full CRUD** (All trials created)	**Full Oversight** (Audit & View All)
Researcher A Authorized Agronomist	Tenant-Isolated	**Read-Only** (Reference & Query)	**Full CRUD** (Own experiments only)	**Strictly Blocked** (Zero Visibility)
Researcher B Authorized Agronomist	Tenant-Isolated	**Read-Only** (Reference & Query)	**Full CRUD** (Own experiments only)	**Strictly Blocked** (Zero Visibility)

3. PostgreSQL Row-Level Security (RLS) Implementation

PostgreSQL enforces security rules directly inside the database kernel. Even if a client performs a broad query `SELECT * FROM trials`, the database engine automatically restricts the result set based on the authenticated session identity (`auth.uid()`):

Table	Action	PostgreSQL Security Rule / Using Expression	Enforcement Effect

`trials`	SELECT	`user_id IS NULL`
		`OR user_id = auth.uid()`
		`OR is_admin()`
			Returns shared baseline trials plus user's private trials. Returns all for `admin1`.

`trials`	INSERT	`WITH CHECK (`
		`  user_id = auth.uid()`
		`  OR is_admin()`
		`)`
			Ensures new trials are tagged with the creator's UUID. Standard users cannot overwrite shared baseline.

`trials`	UPDATE / DELETE	`USING (`
		`  user_id = auth.uid()`
		`  OR is_admin()`
		`)`
			Users can only modify or delete trials they created. `admin1` can modify or delete any.

`harvest_records`
`vegetative_records`
`annual_plot_data`
	ALL	`EXISTS (`
		`  SELECT 1 FROM trials t`
		`  WHERE t.code = trial_code`
		`  AND (t.user_id = auth.uid()`
		`  OR t.user_id IS NULL`
		`  OR is_admin())`
		`)`
			Cascading inherited security: plots, census, and harvest rounds inherit their trial's privacy constraints.

4. Administrator Designation (`admin1`)

To establish `admin1` with system-wide oversight, a secure database helper function `is_admin()` is declared in PostgreSQL. This function checks whether the authenticated user has the admin role or corresponds to the designated administrator email address:

```
-- 1. Helper function to identify administrative authority
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS boolean AS $$
BEGIN
  RETURN (
    auth.jwt() ->> 'email' = 'admin1@mpob.gov.my'
    OR (auth.jwt() -> 'app_metadata' ->> 'role') = 'admin'
  );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- 2. Add creator tracking column to trials
ALTER TABLE public.trials
  ADD COLUMN IF NOT EXISTS user_id uuid REFERENCES auth.users(id),
  ADD COLUMN IF NOT EXISTS created_by_email text;
```

5. Execution Summary

- **Complete Data Protection:** Researchers cannot tamper with or inspect proprietary experiments of colleagues.
- **Shared Institutional Knowledge:** Historical reference trials (DTP1, FT1, etc.) remain permanently available.
- **Centralized Governance:** `admin1` possesses full auditing, modification, and reporting rights across all estates.
- **Export Safety:** All exported CSV files respect Row-Level Security, preventing data leaks at the download stage.